

Python Assignment Examples, Outputs & Simple Notes

Prepared for GitHub Portfolio | Abdul Hafeez

1. What is Python?

Python is a popular programming language used for web development, automation, APIs, data analysis, artificial intelligence, scripting and many other software applications. Its syntax is simple and easy to read.

2. How to Run Python Code

- Install Python from python.org or use an IDE such as VS Code or PyCharm.
- Create a file ending with .py, for example app.py.
- Open a terminal in the same folder.

```
python app.py
```

Explanation: This command runs the Python file. On some systems you may use python3 app.py.

3. Python Syntax Basics

Python uses indentation (spaces at the beginning of a line) to define code blocks. It does not normally use braces {} for blocks.

```
if 10 > 5:  
    print("10 is greater than 5")
```

Expected Output:

```
10 is greater than 5
```

4. Hello World and Variables

```
# 01 - Hello World and Variables  
name = "Abdul Hafeez"  
age = 30  
city = "Riyadh"  
  
print("Hello, Python!")  
print("Name:", name)  
print("Age:", age)  
print("City:", city)
```

Explanation: A variable stores a value. Python automatically understands common value types such as text and numbers.

Expected Output:

```
Hello, Python!  
Name: Abdul Hafeez  
Age: 30  
City: Riyadh
```

5. Basic Data Types

```
# 02 - Basic Data Types
name = "Abdul"      # str
age = 30             # int
salary = 10000.50    # float
is_working = True    # bool

print(type(name))
print(type(age))
print(type(salary))
print(type(is_working))
```

Explanation: str stores text, int stores whole numbers, float stores decimal numbers and bool stores True or False.

Expected Output:

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
```

6. User Input and Calculator

```
# 03 - User Input and Simple Calculator
num1 = float(input("Enter first number: "))
num2 = float(input("Enter second number: "))

print("Addition:", num1 + num2)
print("Subtraction:", num1 - num2)
print("Multiplication:", num1 * num2)
if num2 != 0:
    print("Division:", num1 / num2)
else:
    print("Division: Cannot divide by zero")
```

Explanation: input() receives text from the user. float() converts the entered text into a decimal number. The zero check prevents division by zero.

Expected Output:

```
Enter first number: 10
Enter second number: 5
Addition: 15.0
Subtraction: 5.0
Multiplication: 50.0
Division: 2.0
```

7. Conditions: if / elif / else

```
# 04 - if / elif / else
score = 82

if score >= 90:
    grade = "A"
elif score >= 80:
    grade = "B"
elif score >= 70:
    grade = "C"
else:
    grade = "Needs Improvement"

print("Score:", score)
print("Grade:", grade)
```

Explanation: Conditions allow the program to make decisions. Python checks conditions from top to bottom until one is true.

Expected Output:

```
Score: 82
Grade: B
```

8. Loops

```
# 05 - for and while Loops
print("For loop:")
for number in range(1, 6):
    print(number)

print("While loop:")
count = 1
while count <= 3:
    print(count)
    count += 1
```

Explanation: A for loop repeats over a sequence. A while loop continues while its condition remains true.

Expected Output:

```
For loop:
1
2
3
4
5
While loop:
1
2
3
```

9. Lists

```
# 06 - Lists
skills = ["PHP", "Python", "JavaScript"]
skills.append("PostgreSQL")

print("All skills:", skills)
print("First skill:", skills[0])
print("Total skills:", len(skills))

for skill in skills:
    print("-", skill)
```

Explanation: A list stores multiple values in order. `append()` adds an item and `len()` returns the number of items.

Expected Output:

```
All skills: ['PHP', 'Python', 'JavaScript', 'PostgreSQL']
First skill: PHP
Total skills: 4
- PHP
- Python
- JavaScript
- PostgreSQL
```

10. Dictionaries

```
# 07 - Dictionaries
employee = {
    "name": "Abdul Hafeez",
    "role": "IT Assistant Manager",
    "city": "Riyadh"
}

print(employee["name"])
print(employee["role"])
employee["experience"] = "8+ years"
print(employee)
```

Explanation: A dictionary stores key-value pairs. It is useful for structured information such as an employee or product.

Expected Output:

```
Abdul Hafeez
IT Assistant Manager
{'name': 'Abdul Hafeez', 'role': 'IT Assistant Manager', 'city': 'Riyadh', 'experience': '8+ years'}
```

11. Functions

```
# 08 - Functions
def calculate_total(price, quantity):
    return price * quantity

def greet(name):
    return f"Welcome, {name}!"

print(greet("Abdul"))
print("Order Total:", calculate_total(35, 2))
```

Explanation: A function is reusable code. Parameters receive values and return sends a result back to the caller.

Expected Output:

```
Welcome, Abdul!
Order Total: 70
```

12. Exception Handling

```
# 09 - Exception Handling
try:
    number = int("abc")
    print(number)
except ValueError:
    print("Invalid number. Please enter digits only.")
finally:
    print("Program finished.")
```

Explanation: try runs code that may fail. except handles the error. finally runs whether an error happens or not.

Expected Output:

```
Invalid number. Please enter digits only.
Program finished.
```

13. File Handling

```
# 10 - File Handling
filename = "sample_notes.txt"

with open(filename, "w", encoding="utf-8") as file:
    file.write("Python assignment notes\n")
    file.write("Learning file handling\n")

with open(filename, "r", encoding="utf-8") as file:
    content = file.read()

print(content)
```

Explanation: with open(...) safely opens and closes a file. Mode w writes a file and mode r reads it.

Expected Output:

```
Python assignment notes
Learning file handling
```

14. Object-Oriented Programming (OOP)

```
# 11 - Object-Oriented Programming
class Product:
    def __init__(self, name, price):
        self.name = name
        self.price = price

    def display(self):
        print(f"Product: {self.name} | Price: {self.price} SAR")

product = Product("Chicken Burger", 35)
product.display()
```

Explanation: A class is a blueprint for objects. `__init__` runs when an object is created. `self` refers to the current object.

Expected Output:

```
Product: Chicken Burger | Price: 35 SAR
```

15. List Comprehension

```
# 12 - List Comprehension
numbers = [1, 2, 3, 4, 5]
squares = [number ** 2 for number in numbers]
even_numbers = [number for number in numbers if number % 2 == 0]

print("Squares:", squares)
print("Even numbers:", even_numbers)
```

Explanation: List comprehension is a short way to create a new list from an existing sequence.

Expected Output:

```
Squares: [1, 4, 9, 16, 25]
Even numbers: [2, 4]
```

16. Modules and Standard Library

```
# 13 - Standard Library Modules
import math
from datetime import date

print("Square root of 81:", math.sqrt(81))
print("Today:", date.today())
```

Explanation: Modules provide reusable functionality. `math` contains mathematical functions and `datetime` provides date/time tools.

Expected Output:

```
Square root of 81: 9.0
Today: YYYY-MM-DD
```

17. Mini Assignment: Restaurant Order Calculator

```
# 14 - Mini Assignment: Restaurant Order Calculator
menu = {
    "burger": 35.0,
    "pasta": 45.0,
    "juice": 18.0
}

order = [
    {"item": "burger", "quantity": 2},
    {"item": "juice", "quantity": 1}
]

subtotal = 0
for row in order:
    price = menu[row["item"]]
    line_total = price * row["quantity"]
    subtotal += line_total
    print(f'{row["item"].title()} x {row["quantity"]}: {line_total:.2f} SAR')

vat_rate = 0.15
vat = subtotal * vat_rate
grand_total = subtotal + vat

print(f"Subtotal: {subtotal:.2f} SAR")
print(f"VAT (15%): {vat:.2f} SAR")
print(f"Grand Total: {grand_total:.2f} SAR")
```

Explanation: This mini project combines dictionaries, lists, loops, calculations and formatted output. It also calculates 15% VAT.

Expected Output:

```
Burger x 2: 70.00 SAR
Juice x 1: 18.00 SAR
Subtotal: 88.00 SAR
VAT (15%): 13.20 SAR
Grand Total: 101.20 SAR
```

18. Tuples

```
cities = ("Riyadh", "Jeddah", "Dammam")
print(cities[0])
print(len(cities))
```

Explanation: A tuple is ordered like a list but is normally used when values should not be changed.

Expected Output:

```
Riyadh
3
```

19. Sets

```
skills = {"Python", "PHP", "Python", "SQL"}
print(skills)
```

Explanation: A set stores unique values. Duplicate Python is removed. The displayed order can vary.

Expected Output:

```
{'Python', 'PHP', 'SQL'} # order may vary
```

20. String Methods

```
text = " python developer "  
print(text.strip().title())  
print("developer" in text)
```

Explanation: strip() removes outside spaces, title() changes words to title case, and in checks membership.

Expected Output:

```
Python Developer  
True
```

21. Useful Built-in Functions

```
numbers = [10, 20, 30]  
print(len(numbers))  
print(sum(numbers))  
print(max(numbers))  
print(min(numbers))
```

Explanation: Python provides useful built-in functions for common operations.

Expected Output:

```
3  
60  
30  
10
```

End of Python Assignment & Notes